

Group 7

Charlie Kopp, Isaiah Mireles, Joel Yoon

FiNGERHUT®

Data

failure ongoing success

The Data

Three types of journeys

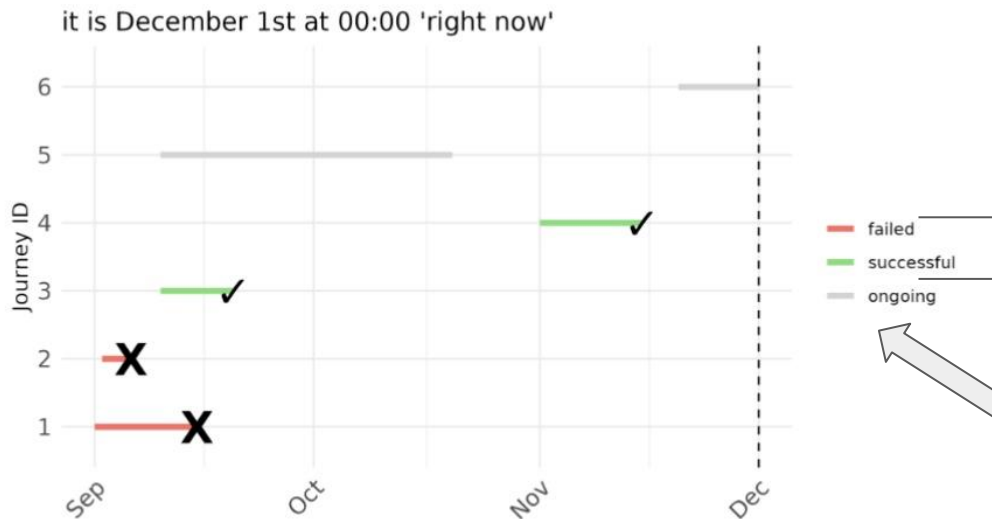
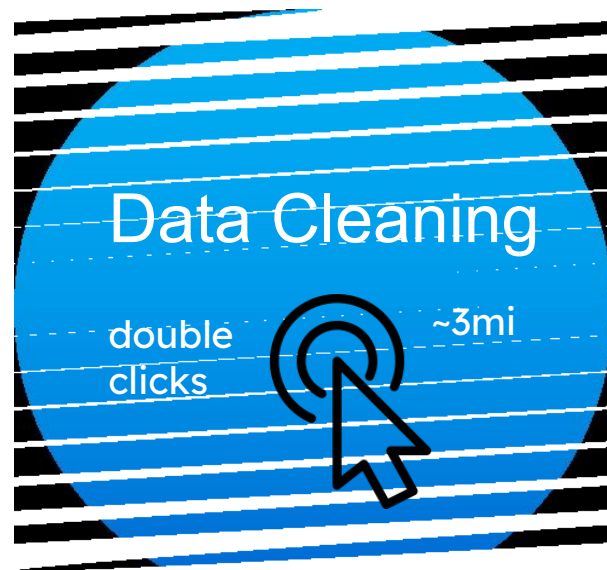


Figure: There are completed and ongoing journeys, for the completed ones we know whether they were successful or failed.



Training

Testing

Flatten

reduce **id** paths
to **id** prob

&

Feature
Engineer



Temporal

first_action_ts
last_action_ts
journey_length_s
days_inactive
total_actions
mean_gap_sec
median_gap_sec
max_gap_sec

Temporal **variation** in

Successful journey usually
take less time than a
“Unsuccessful”



Approval

n_account_downpayment

cleared

n_account_downpayment

received

n_application_web

approved

n_application_web

declined

n_application_phone

approved

n_application_phone

declined

Website Activity

n_account_activation

n_begin_checkout

n_browse_products

n_add_to_cart

N_view_cart

n_application_web_submit

n_application_web_view

Promotion

n_campaignemail_clicked

n_catalog_email_experian

n_catalog_mail

n_campaign_click

n_fingerhut_university

Final Behavior

n_place_downpayment

n_place_order_phone

n_place_order_web n_pre-application_3rd party affiliates

n_promotion_created n_site_registration

- Approval **variation**
- Promotion **variation**
- Website Activity **variation**
- Final Behavior **variation**



Approval

Promotion

Website Activity

Final Behavior

Training

Sample

The Data

Three types of journeys

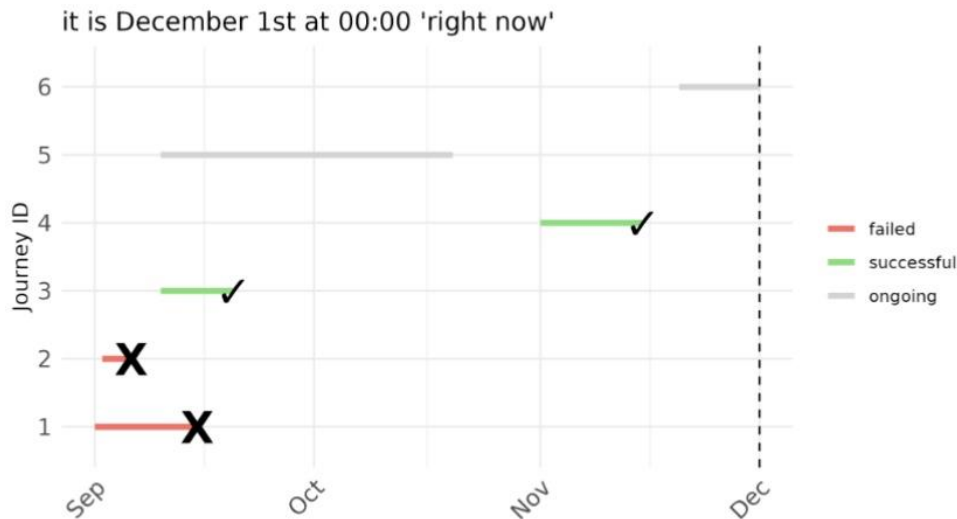


Figure: There are completed and ongoing journeys, for the completed ones we know whether they were successful or failed.

Sampling Methods

Simple random sample
sample randomly within a completed (s/f) journey

Sample proportional
total_actions

More Actions contribute more training snapshots than short journeys.

Sample proportional
journey_days

Longer journeys contribute more training snapshots than short journeys.

Prophet package

What is the Prophet package

- Open source forecasting tool developed by Meta in 2017
- Used for reliable forecasts, especially in terms of business data with seasonality and holidays
- Works well with limited hyperparameter tuning

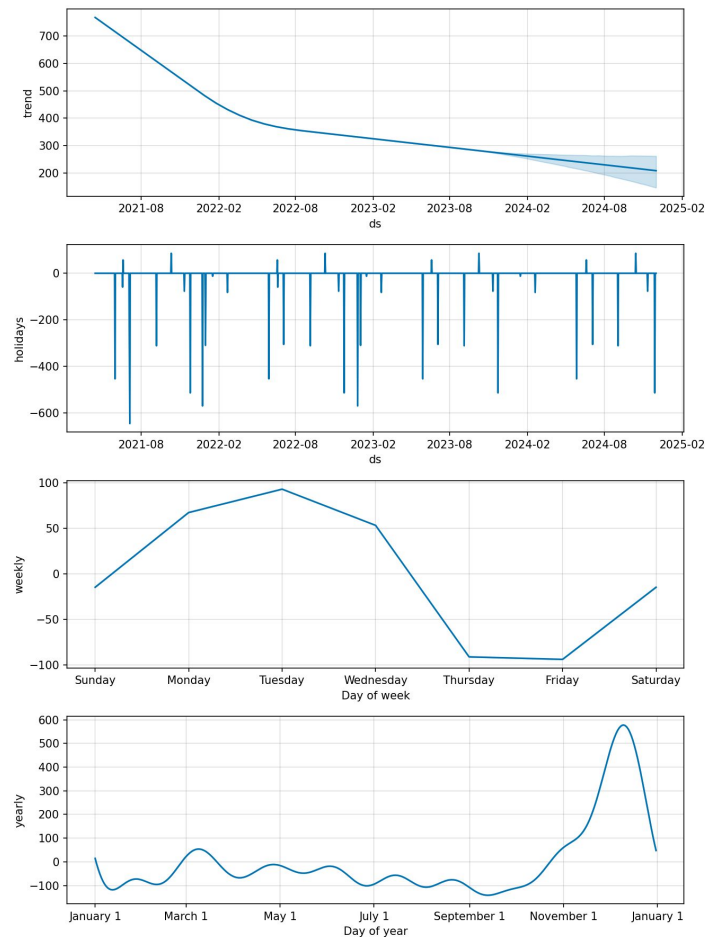


Prophet: calendar signal

- Question: can time of year, week, or holiday season in which a journey takes place tell us anything useful?
- Prophet models order-shipped volume over calendar time.
- XGBoost still handles the journey-level prediction; Prophet supplies seasonality context.

Prophet Components Graph

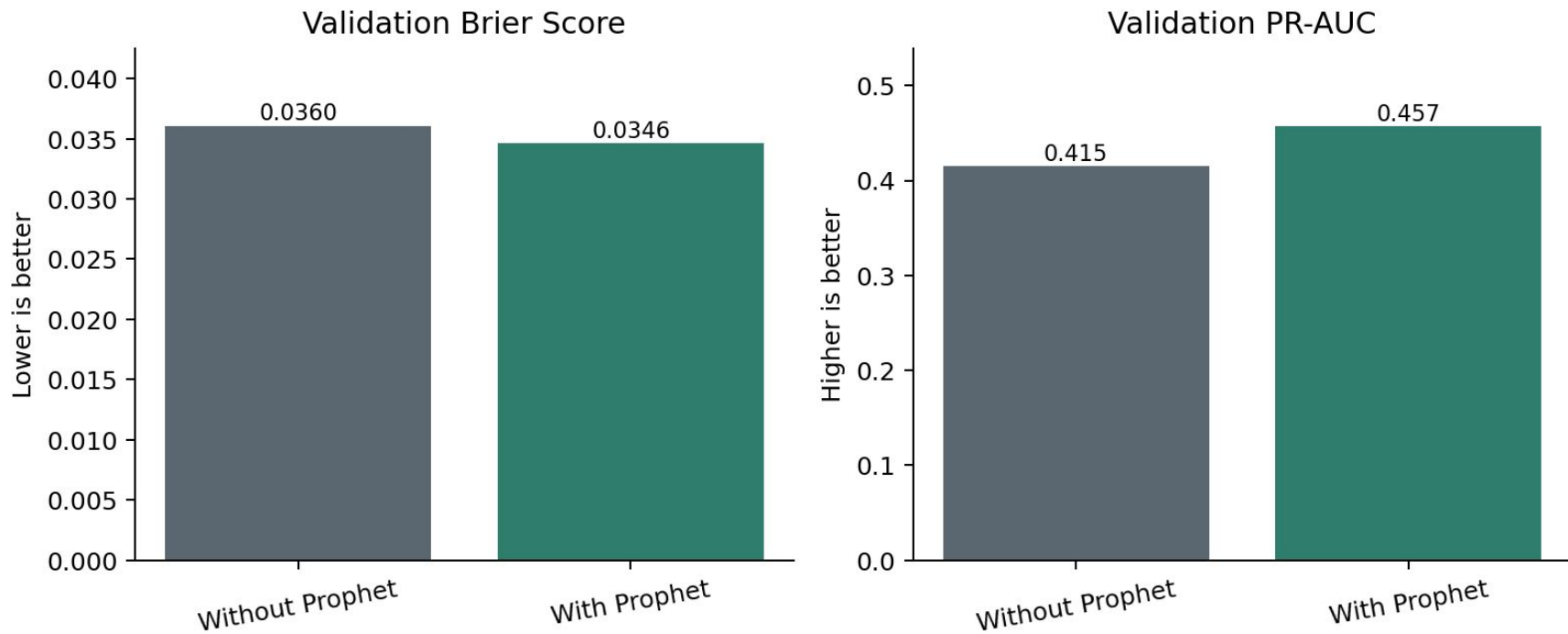
- Trend shows long-term shipped-order volume
- Holiday panel captures US holiday effects
- Weekly panel shows weekday shipping pattern
- Yearly panel captures broad seasonal movement, especially late-year spikes



What We Actually Compared

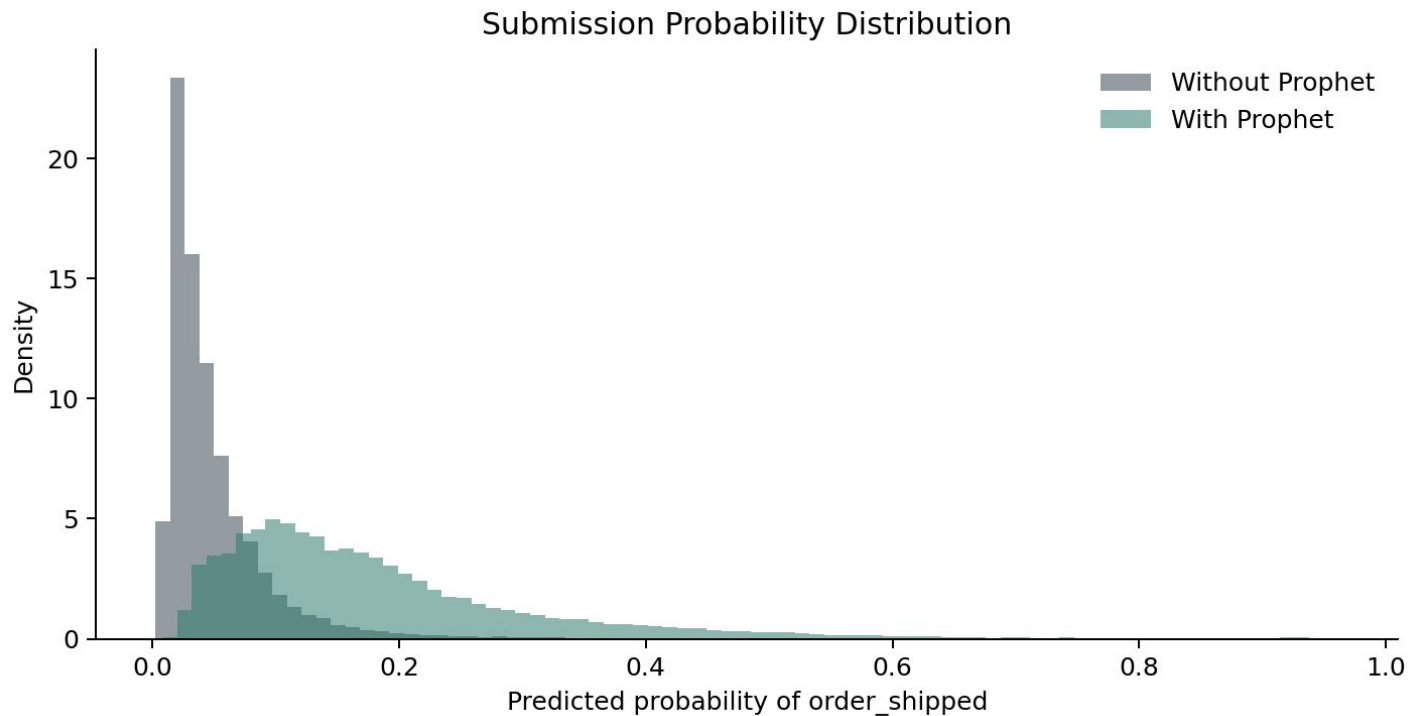
- Same XGBoost journey-level model in both runs
- Same train/validation split
- Without Prophet: journey/event features only
- With Prophet: journey/event features + Prophet calendar features
- Prophet features: trend, US holidays, weekly cycle, yearly cycle

Validation Results Graph



Prophet improved both validation metrics: lower Brier and higher PR-AUC

Caveat / Distribution Shift



Without Prophet mean = 0.056

With Prophet mean = 0.184

RNN (LSTM) fitting

Why a Recurrent Neural Network?

- prior logistic and tree-based prediction models rely on ‘flattened’, tabular data
- this restricts us to using 1D summary indicators (counts, avg. time gap, etc.)
- clearly, as shown by our Prophet model, time has a significant impact on purchasing behavior
 - but Prophet models don’t incorporate external information like event types, browsing frequency, the path a user takes, etc. Most importantly, it does not forecast individual-level data.
- Hence, **recurrent neural networks** allow us to train and make predictions on unflattened (event-level) data, taking each user’s individual path and patterns into account

RNN (LSTM) Implementation

- using the `keras3` package
 - available in Python and R
- format data: take 100 most recent actions, their time gaps and pad short journeys
- map event_name to embedding layer (27 unique actions → 14D) and time_gap to 4-neuron dense layer
- n=64 hidden units for LSTM



Keras

RNN (LSTM) Evaluation

- Brier score = 0.0425
 - worse than tree-based models at ~ 0.035
 - no feature engineering required
- Processing time for training + prediction
- Interpretability